NAMA : ARINA TRI YUNI WAHYUNING TIYAS
NIM : 23031554203

Pertanyaan Pendahuluan (Checkpint)

1. Apa nama database yang digunakan pada tutorial ini? neo4j, database tersebut diakses melalui Python Driver menggunakan URI neo4j://localhost:7687 dan autentikasi sesuai konfigurasi Neo4j yang digunakan.
2. Sebutkan akses lain (seperti sintaks DDL/DML CYPHER) yang dapat digunakan mengakses database lain dalam 1 instance! selain dari python, database Neo4j bisa diakses melalui query cyper dimana cyper dapat mendukung operasi untuk membuat, membaca, memperbaharui, dan menghapus data.
3. Apa jenis graf pertemanan yang telah dibuat di sini? Graf pertemanan yang dibuat merupakan graf berarah atau directed graph. Graf tersebut menggunakan node berlabel Person dan relasi KNOWS. Arah relasi menunjukkan bahwa seseorang mengenal orang lain, sehingga relasi dari Arthur ke Guinevere tidak otomatis berarti Guinevere mengenal Arthur

2.1 Wrangling dan EDA (Maks. skor: 45)

1. Tentukan 2 periode waktu sebagai filter dari data interaksi. Kombinasi periode yg dipilih per individu tidak boleh sama dalam 1 kelas.
   Misal: Fold training (P1) → 01/2023 - 03/2023; Fold testing (P2) 01/2024 - 06/2024.
   Pada penelitian ini, saya menggunakan dataset RetailRocket yang berisi aktivitas pengunjung terhadap produk. Dataset dibagi menjadi dua periode berdasarkan timestamp, yaitu P1 sebagai data training dan P2 sebagai data testing.

Periode	Rentang waktu UTC	Kegunaan
P1	3 Mei 2015 – 31 Agustus 2015	Training
P2	1 September 2015 – 18 September 2015	Testing

   Pembagian waktu dilakukan agar model dibangun menggunakan data yang terjadi lebih awal, kemudian dievaluasi menggunakan data pada periode berikutnya. Dengan demikian, data testing tidak digunakan untuk membangun model rekomendasi

2. Terapkan proses filter ini pada timestamp pada proses persiapan data, lalu simpan pada sebuah file .csv!
   Dataset RetailRocket memiliki lima kolom, yaitu timestamp, visitorid, event, itemid, dan transactionid. Kolom timestamp menggunakan format Unix epoch dalam milidetik, sehingga perlu dikonversi menjadi format tanggal dan waktu agar proses pembagian periode lebih mudah dilakukan. Hasil pemeriksaan menunjukkan bahwa dataset awal terdiri dari 2.756.101 baris dan tidak memiliki nilai kosong pada kolom timestamp, visitorid, event, serta itemid. Kolom transactionid memiliki 2.733.644

nilai kosong karena ID transaksi hanya tersedia pada event transaksi. Setelah dilakukan pembagian periode, diperoleh 2.452.391 event pada P1 dan 303.710 event pada P2. Data hasil filter disimpan dalam file CSV untuk digunakan pada proses berikutnya.

Kode untuk menyimpan data:

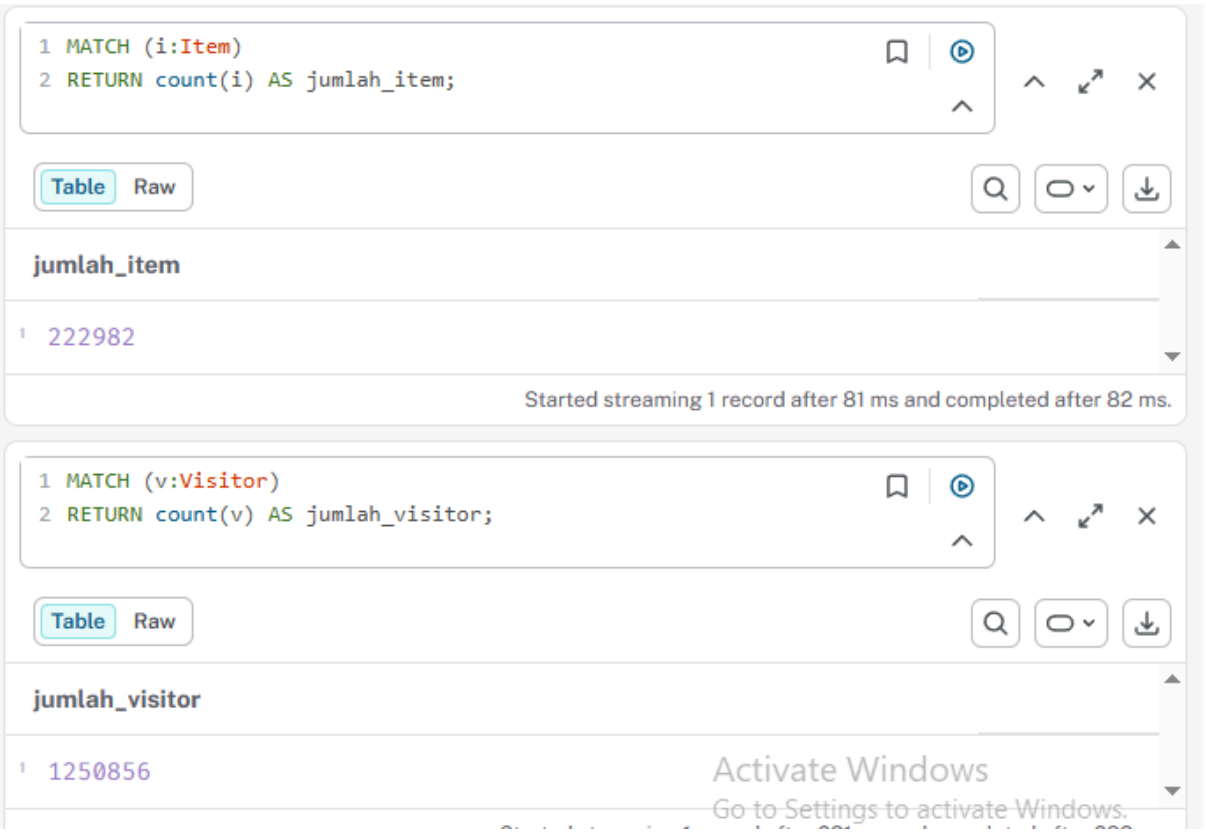
```
# Simpan CSV
p1.drop(columns=["datetime"], errors="ignore").to_csv(
    "events_P1_training.csv", index=False
)

p2.drop(columns=["datetime"], errors="ignore").to_csv(
    "events_P2_testing.csv", index=False
)
```

3. Lakukan query untuk mendapatkan komposisi dari subgraph yang dibuat. Uraikan jenis simpul dan jumlah masing-masing simpul dengan Python API! Jelaskan CYPHER SQL yang digunakan!

   Saya membangun graf RetailRocket menggunakan dua jenis node, yaitu Visitor dan Item. Node Visitor merepresentasikan pengunjung, sedangkan node Item merepresentasikan produk yang dilihat atau dibeli oleh pengunjung. Interaksi antara pengunjung dan produk direpresentasikan melalui tiga jenis relasi, yaitu VIEWED, ADDED_TO_CART, dan BOUGHT.

   Hasil impor data P1 adalah sebagai berikut.



```
MATCH ()-[r]->()
RETURN type(r) AS jenis_relasi,
       count(r) AS jumlah
ORDER BY jumlah DESC;
```

Table Raw

	jenis_relasi	jumlah
1	"VIEWED"	2370652
2	"ADDED_TO_CART"	61651
3	"BOUGHT"	20088

Started streaming 3 records after 336 ms and completed after 2,244 ms.

Activate Windows

Query MATCH digunakan untuk mencocokkan pola graf, RETURN menentukan data yang ditampilkan, count() menghitung jumlah node atau relasi, dan ORDER BY mengurutkan hasil berdasarkan jumlahnya. Berdasarkan hasil tersebut, relasi VIEWED merupakan relasi yang paling banyak ditemukan. Hal ini sesuai dengan karakteristik dataset e-commerce, karena aktivitas melihat produk lebih sering terjadi dibandingkan aktivitas menambahkan produk ke keranjang atau melakukan transaksi.

4. Lakukan Vector Space Modelling (VSM) terhadap simpul yang diasumsikan sebagai User. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!
   Pada tahap ini, saya membentuk User–Item Vector Space Model untuk merepresentasikan interaksi setiap pengunjung terhadap produk. Setiap baris mewakili satu user dan setiap kolom mewakili satu item. Nilai pada matriks menunjukkan keberadaan interaksi user terhadap item sesuai representasi yang digunakan.
   Hasil pembentukan VSM adalah sebagai berikut.

```
Ukuran matriks User-Item: (1250856, 222982)
Jumlah interaksi user-item unik: 1905695
Jumlah pengguna: 1250856
Jumlah item: 222982
Kepadatan matriks: 0.000683%
```

   Kepadatan matriks yang sangat rendah menunjukkan bahwa sebagian besar user hanya berinteraksi dengan sebagian kecil item yang tersedia. Oleh karena itu, matriks bersifat sparse dan lebih efisien direpresentasikan menggunakan sparse matrix daripada matriks padat.
   VSM ini digunakan untuk menghitung kemiripan antar-user berdasarkan pola interaksi terhadap item.

5. Lakukan VSM terhadap simpul yang diasumsikan sebagai Item. Uraikan bagaimana karakteristiknya (dimensi, shape, plot networkx, dst.)!

Selain melakukan proyeksi berdasarkan user, saya juga melakukan proyeksi berdasarkan item. Pada proyeksi ini, setiap item direpresentasikan menggunakan pola interaksi user yang berhubungan dengan item tersebut. Matriks Item–User diperoleh dengan melakukan transpose terhadap matriks User–Item. Dengan demikian, shape matriks berubah dari (1250856, 222982) menjadi (222982, 1250856).
Kode yang digunakan:

```
# Baris = pengguna, kolom = item
user_vsm = user_item_matrix

# Baris = item, kolom = pengguna
item_vsm = user_item_matrix.T.tocsr()

print("User VSM:", user_vsm.shape)
print("Item VSM:", item_vsm.shape)
```

Proyeksi Item–Item digunakan untuk mengidentifikasi produk yang memiliki pola interaksi serupa. Dua item dianggap mirip apabila keduanya memiliki pola pengunjung yang berinteraksi secara serupa.

6. Berdasarkan eksplorasi no. 4 dan 5, uraikan proyeksi manakah yang lebih baik (efektif/efisien)!
   Berdasarkan hasil eksplorasi, saya membandingkan proyeksi User–User dan Item–Item. Jumlah user pada dataset lebih besar daripada jumlah item, sehingga proyeksi Item–Item memiliki jumlah entitas yang lebih sedikit.
   Hasil perbandingan similarity pada sampel adalah sebagai berikut.

```
USER SIMILARITY
Jumlah pasangan similarity > 0: 4594
Jumlah pasangan similarity >= 0.1: 174
Similarity maksimum antar-user: 0.24760044495126168

ITEM SIMILARITY
Jumlah pasangan similarity > 0: 2732
Jumlah pasangan similarity >= 0.1: 16
Similarity maksimum antar-item: 0.1939075940138191
```

   Berdasarkan hasil tersebut, saya memilih proyeksi User–User untuk eksperimen rekomendasi karena pendekatan yang digunakan adalah User-Based Collaborative Filtering. Pendekatan ini mencari pengguna yang memiliki pola interaksi serupa untuk menghasilkan rekomendasi kepada pengguna target. Meskipun demikian, Item–Item memiliki keunggulan dari sisi jumlah entitas yang lebih sedikit. Efisiensi komputasi secara keseluruhan tetap bergantung pada jumlah relasi, struktur matriks, dan metode pencarian similarity yang digunakan.

7. Berdasarkan analisis no. 6, lakukan perhitungan similarity pada hasil graf yg terproyeksi dengan menggunakan Python

(a) Tuliskan CYPHER SQL untuk mendapatkan 5 records dengan similarity tertinggi (Anda tentukan sendiri record acuannya)!

Saya menggunakan cosine similarity untuk mengukur kemiripan antara dua user. Berdasarkan hasil perhitungan pada sampel 100 user aktif, diperoleh 4.594 pasangan dengan similarity positif dan 174 pasangan dengan similarity minimal 0,1. Nilai similarity maksimum antar-user berbeda adalah 0,24760044495126168. Untuk mendapatkan lima relasi dengan similarity tertinggi dari database Neo4j, saya menggunakan query berikut:

```
MATCH (u1:Visitor)-[r:SIMILAR_TO]->(u2:Visitor)
RETURN u1.visitorId, u2.visitorId, r.similarity
ORDER BY r.similarity DESC
LIMIT 5;
```

	u1.visitorId	u2.visitorId	r.similarity
1	1150086	530559	0.24760044495126168
2	1119917	917504	0.24669439082546613
3	518659	1336835	0.20952201578026575
4	1150086	371606	0.20002859838249246
5	530559	371606	0.18811030309875296

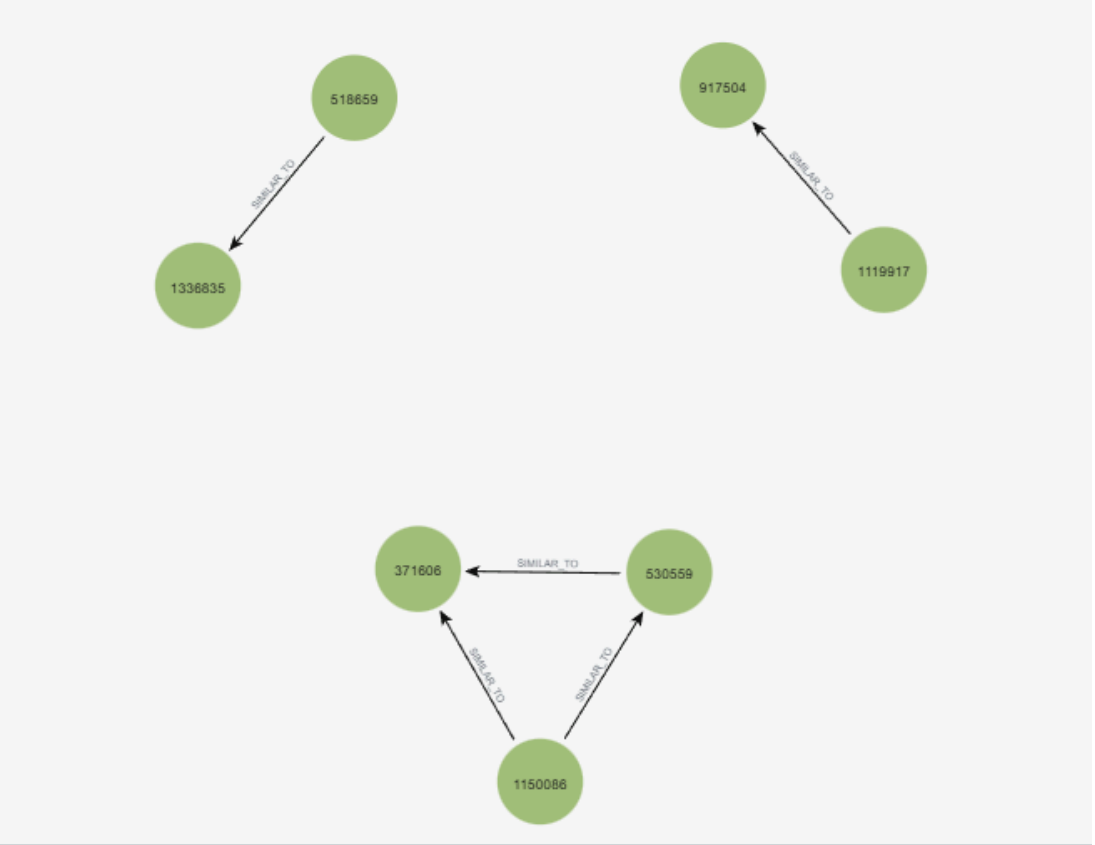
Query tersebut mengambil lima relasi SIMILAR_TO dengan nilai similarity terbesar. Nilai yang ditampilkan merupakan nilai yang benar-benar tersimpan di database, sehingga dapat digunakan untuk memeriksa hasil perhitungan similarity.

(b) Tuliskan kembali nilai similarity-similarity tersebut pada graf yang telah ada! Perlihatkan hasil querynya!

Untuk menampilkan relasi beserta node yang terhubung, saya menggunakan query:

```
MATCH (u1:Visitor)-[r:SIMILAR_TO]->(u2:Visitor)
WHERE r.similarity IS NOT NULL
RETURN u1, r, u2
ORDER BY r.similarity DESC
LIMIT 5;
```

Query tersebut menghasilkan visualisasi hubungan antar-user pada Neo4j Browser. Relasi SIMILAR_TO menunjukkan bahwa dua user memiliki pola interaksi yang serupa, sedangkan properti similarity menunjukkan tingkat kemiripannya.



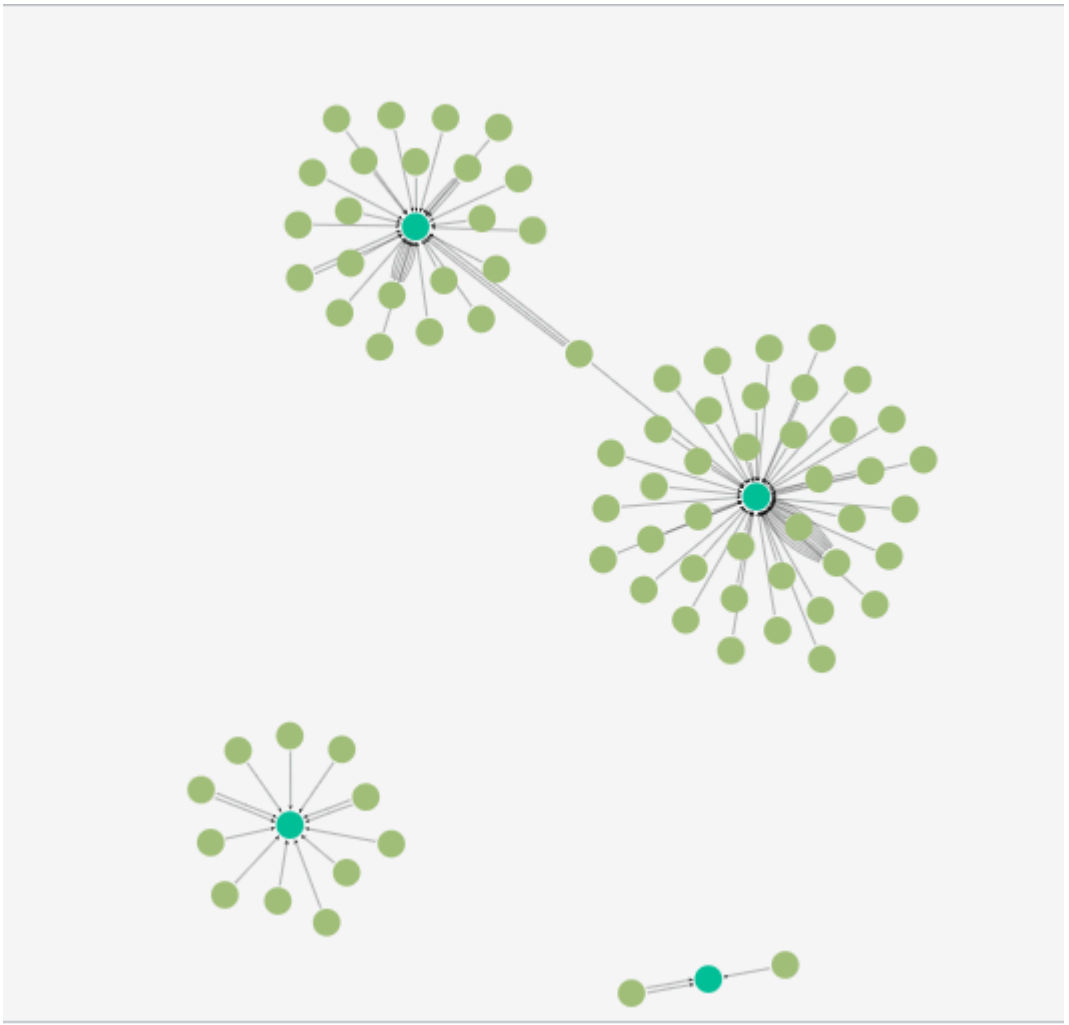
Berdasarkan hasil sebelumnya, sebanyak 174 relasi User–User dengan similarity minimal 0,1 telah berhasil disimpan ke Neo4j.

8. Tuliskan CYPHER SQL dan tunjukkan hasil graf-nya untuk memperlihatkan kondisi: sebelum dan sesudah perhitungan similarity!

Sebelum perhitungan similarity dilakukan, graf hanya merepresentasikan interaksi pengunjung terhadap produk. Setelah perhitungan similarity, graf juga memiliki relasi yang menunjukkan kemiripan antar-user.

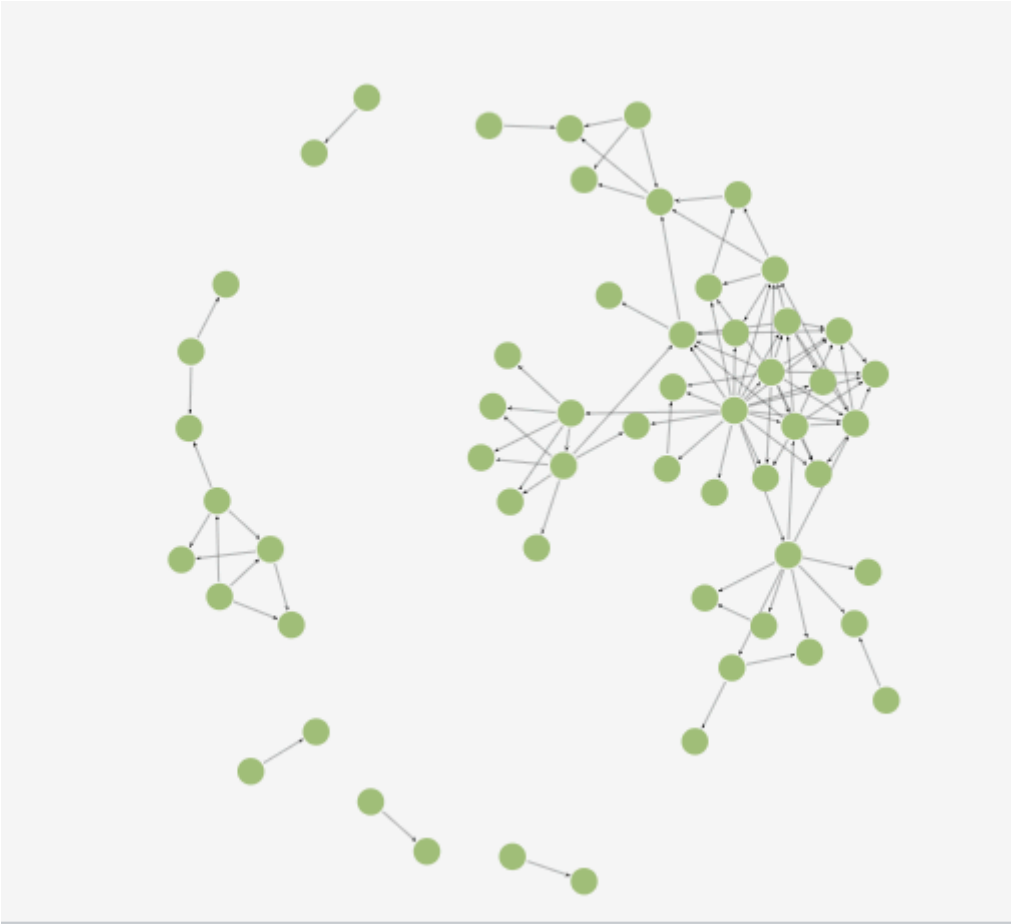
Query untuk menampilkan graf sebelum penambahan relasi similarity:

```
MATCH (u:Visitor)-[r]->(i:Item)
WHERE type(r) IN ['VIEWED', 'ADDED_TO_CART', 'BOUGHT']
RETURN u, r, i
LIMIT 100;
```



Query untuk menampilkan graf setelah penambahan relasi similarity:

```
1
2 MATCH (u1:Visitor)-[r:SIMILAR_TO]->(u2:Visitor)
3 WHERE r.similarity >= 0.1
4 RETURN u1, r, u2
5 ORDER BY r.similarity DESC
6 LIMIT 100;
7
```



Perbedaan kedua graf menunjukkan bahwa graf awal digunakan untuk merepresentasikan perilaku pengunjung, sedangkan graf hasil pengembangan memiliki informasi tambahan berupa kemiripan antar-user. Informasi similarity tersebut digunakan untuk mendukung sistem rekomendasi.

2.2 Analytics (Maks. skor: 55)

1. Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
   (a) tuliskan skrip Python dalam merekomendasikan KNN-nya!
   Pada tahap Analytics, saya menggunakan metode User-Based Collaborative Filtering dengan KNN. Metode ini bekerja dengan mencari lima user yang memiliki cosine similarity tertinggi terhadap user target. Item yang pernah diinteraksikan oleh tetangga kemudian diberi skor berdasarkan nilai similarity.

```
neighbor_order = np.argsort(user_sim[target_pos])[::-1]

neighbors = []
for pos in neighbor_order:
    candidate_idx = int(user_idx[pos])

    if candidate_idx == example_idx:
        continue

    sim = float(user_sim[target_pos, pos])
    if sim <= 0:
        continue

    neighbors.append({
        "visitorid": int(user_ids[candidate_idx]),
        "similarity": sim,
        "matrix_index": candidate_idx
    })

    if len(neighbors) == 5:
        break

neighbors_df = pd.DataFrame(neighbors)
display(neighbors_df)

print("Jumlah tetangga yang ditemukan:", len(neighbors_df))
```

Setelah mendapatkan lima tetangga, saya menghasilkan tiga rekomendasi menggunakan fungsi rekomendasi yang sudah dibuat pada tahap sebelumnya.

```
from collections import defaultdict

def recommend_for_sample_position(target_sample_pos, n_recommendations=5):
    target_matrix_idx = int(user_idx[target_sample_pos])
    similarity_row = user_sim[target_sample_pos]

    neighbor_positions = np.argsort(similarity_row)[::-1]
    scores = defaultdict(float)
    used_neighbors = 0

    for pos in neighbor_positions:
        neighbor_matrix_idx = int(user_idx[pos])

        if neighbor_matrix_idx == target_matrix_idx:
            continue

        similarity = float(similarity_row[pos])
        if similarity <= 0:
            continue

        # Tambahkan skor similarity untuk setiap item milik tetangga
        neighbor_items = user_vsm[neighbor_matrix_idx].indices
        for item_matrix_idx in neighbor_items:
            scores[int(item_matrix_idx)] += similarity

        used_neighbors += 1
        if used_neighbors == 5:
            break
```

```
    # Hilangkan item yang sudah pernah diinteraksikan target
    seen_items = set(user_vsm[target_matrix_idx].indices)
    ranked_items = [
        (item_idx, score)
        for item_idx, score in scores.items()
        if item_idx not in seen_items
    ]
    ranked_items.sort(key=lambda x: x[1], reverse=True)

    return [
        {
            "itemid": int(item_ids[item_idx]),
            "score": float(score)
        }
        for item_idx, score in ranked_items[:n_recommendations]
    ]

recommendation_df = pd.DataFrame(
    recommend_for_sample_position(target_pos, n_recommendations=3)
)

print("Rekomendasi untuk Visitor ID:", example_user_id)
display(recommendation_df)
```

(b) berikan 3 item yang diiklankan untuk k = 5 pada penentuan KNN nya!

Hasil rekomendasi diperoleh berdasarkan skor tertinggi dari item yang pernah diinteraksikan oleh lima tetangga terdekat, dengan mengecualikan item yang sudah pernah diinteraksikan oleh user target. Tabel rekomendasi yang digunakan dalam laporan adalah sebagai berikut. Nilai ID item dan skor harus diisi menggunakan output aktual dari recommendation_df.

Rekomendasi untuk Visitor ID: 1150086

	itemid	score
0	254052	0.976875
1	346648	0.806268
2	211486	0.806268

2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:

(a) Hit@5

Hit@5 digunakan untuk mengetahui apakah setidaknya satu item relevan muncul dalam lima rekomendasi teratas.

Hasil evaluasi yang saya peroleh adalah sebagai berikut.

- Jumlah user yang berhasil dievaluasi: 30.
- Rata-rata Hit@5: 0,23333333333333334.

Nilai tersebut menunjukkan bahwa 7 dari 30 user memiliki setidaknya satu item relevan dalam lima rekomendasi teratas. Dengan demikian, sistem rekomendasi berhasil menemukan item relevan pada sebagian pengguna yang diuji.

(b) NDCG@5

NDCG@5 digunakan untuk mengukur kualitas urutan rekomendasi dengan mempertimbangkan posisi item relevan. Item relevan yang berada pada peringkat lebih tinggi memberikan kontribusi lebih besar dibandingkan item relevan yang berada pada peringkat lebih rendah. Hasil evaluasi yang saya peroleh adalah rata-rata NDCG@5 sebesar 0,05536132649087945. Nilai NDCG@5 tersebut menunjukkan bahwa kualitas pemeringkatan rekomendasi masih terbatas. Walaupun beberapa pengguna mendapatkan item relevan, item relevan belum selalu berada pada posisi peringkat yang tinggi.

denga code sebagai berikut

```python
def calculate_hit_ndcg(recommended_items, relevant_items, k=5):
    ranked = recommended_items[:k]

    # Hit@K = 1 jika ada minimal satu item relevan
    hit = int(any(item in relevant_items for item in ranked))

    # NDCG@K dengan relevansi biner
    dcg = sum(
        1.0 / np.log2(rank + 2)
        for rank, item in enumerate(ranked)
        if item in relevant_items
    )

    ideal_hits = min(len(relevant_items), k)
    idcg = sum(
        1.0 / np.log2(rank + 2)
        for rank in range(ideal_hits)
    )

    ndcg = dcg / idcg if idcg > 0 else 0.0
    return hit, ndcg
```

```
evaluation_rows = []

# Gunakan user yang sama dengan sampel similarity
sample_user_ids = [int(user_ids[idx]) for idx in user_idx]

for sample_pos, visitor_id in enumerate(sample_user_ids):
    if visitor_id not in ground_truth:
        continue

    target_matrix_idx = int(user_idx[sample_pos])
    seen_items = set(map(int, user_vsm[target_matrix_idx].indices))
    # Konversi indeks kolom VSM ke item ID yang sebenarnya
    seen_item_ids = {int(item_ids[i]) for i in seen_items}

    relevant_items = ground_truth[visitor_id] - seen_item_ids

    # Ground truth kosong tidak memberi informasi untuk evaluasi ranking
    if not relevant_items:
        continue

    recs = recommend_for_sample_position(
        sample_pos,
        n_recommendations=5
    )
    recommended_items = [row["itemid"] for row in recs]
```

```
    hit5, ndcg5 = calculate_hit_ndcg(
        recommended_items,
        relevant_items,
        k=5
    )

    evaluation_rows.append({
        "visitorid": visitor_id,
        "n_relevant_test_items": len(relevant_items),
        "n_recommended": len(recommended_items),
        "Hit@5": hit5,
        "NDCG@5": ndcg5
    })

evaluation_df = pd.DataFrame(evaluation_rows)

display(evaluation_df.head(10))

if len(evaluation_df) > 0:
    print("Jumlah user yang berhasil dievaluasi:", len(evaluation_df))
    print("Rata-rata Hit@5:", evaluation_df["Hit@5"].mean())
    print("Rata-rata NDCG@5:", evaluation_df["NDCG@5"].mean())
else:
    print(
        "Belum ada user dengan ground truth P2 yang memenuhi kriteria. "
        "Periksa cakupan user dan item."
    )
```

dan hasil sebagai berikut

	visitorid	n_relevant_test_items	n_recommended	Hit@5	NDCG@5
0	1150086	456	5	0	0.000000
1	530559	206	5	0	0.000000
2	895999	187	5	1	0.339160
3	152963	587	5	0	0.000000
4	316850	76	5	0	0.000000
5	79627	413	5	0	0.000000
6	638482	216	5	0	0.000000
7	310759	99	5	1	0.213986
8	229157	9	5	0	0.000000
9	247235	235	5	0	0.000000

```
Jumlah user yang berhasil dievaluasi: 30
Rata-rata Hit@5: 0.23333333333333334
Rata-rata NDCG@5: 0.05536132649087945
```